Université Mohammed Premier
École Nationale de l'Intelligence Artificielle
et du Digital — Berkane

COMPTE RENDU DE TRAVAUX PRATIQUES

TESTING LOGICIEL

Tests unitaires, tests d'intégration et isolation des dépendances

TP 2

RÉALISÉ PAR **Zakariae El Haddouchi**

FILIÈRE Intelligence Artificielle (cycle ingénieur)

MODULE DevOps

PROFESSEUR **Habchi Ali**

CODE SOURCE

github.com/Eng-ZAKARIAE-AI/devops-module/tree/main/testing/tp-2-testing

Année universitaire 2026 – 2027

Exercice 1 | Tests unitaires

`tests/unit/test_livraison.py`

```python
import pytest

from app.livraison import frais_livraison


def test_livraison_panier_ordinaire():
    assert frais_livraison(120) == 25


def test_livraison_panier_299():
    assert frais_livraison(299) == 25


def test_livraison_panier_300():
    assert frais_livraison(300) == 0


def test_livraison_panier_301():
    assert frais_livraison(301) == 0


def test_livraison_panier_zero():
    assert frais_livraison(0) == 25


def test_livraison_montant_negatif():
    with pytest.raises(ValueError):
        frais_livraison(-1)
```

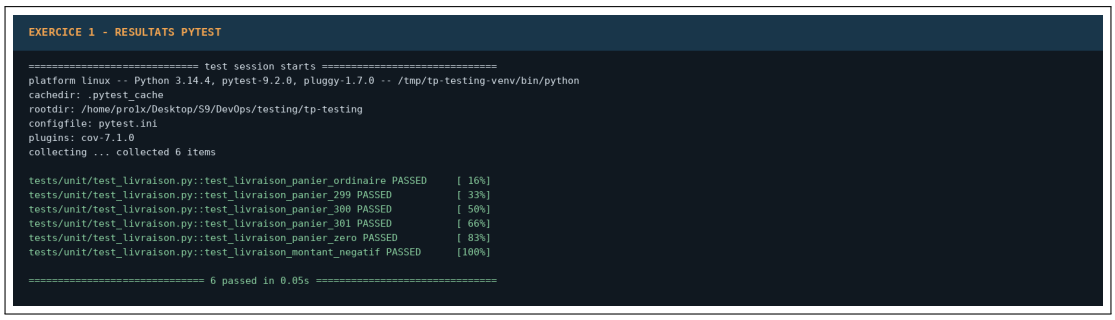


Fig. 1 : Sortie de l'exercice 1

Interprétation des résultats. Les tests vérifient les montants de part et d'autre du seuil, le panier vide et le rejet d'un montant négatif. Le cas de 300 DH a révélé le défaut : l'ancienne condition `montant > 300` facturait encore 25 DH au seuil. La règle a été corrigée en `montant >= 300` ; les frais sont donc gratuits à partir de 300 DH inclus. Une exception `ValueError` attendue compte comme un test réussi.

Réponse à la question. Tester seulement 120 DH et 500 DH n'aurait pas révélé l'erreur : ces valeurs se trouvent de part et d'autre du seuil et donnent respectivement 25 DH et 0 DH avec les deux conditions (> et >=). Le test à 300 DH est nécessaire pour distinguer précisément ces conditions.

Exercice 2 | Nettoyer un texte avant son analyse

app/texte.py

```python
def nettoyer_texte(texte):
  if not isinstance(texte, str):
    raise TypeError("Texte attendu")
  return " ".join(texte.lower().split())
```

tests/test_texte.py

```python
import pytest

from app.texte import nettoyer_texte


@pytest.mark.parametrize(
  "entree, attendu",
  [
    (" BONJOUR monde ", "bonjour monde"),
    (" déjà propre ", "déjà propre"),
    ("", ""),
    (" ", ""),
    (" PYTHON ", "python"),
  ],
)
def test_nettoyer_texte(entree, attendu):
  assert nettoyer_texte(entree) == attendu


@pytest.mark.parametrize("entree", [123, None])
def test_nettoyer_texte_entree_non_textuelle(entree):
  with pytest.raises(TypeError):
    nettoyer_texte(entree)


def test_nettoyer_texte_deja_nettoye_ne_change_pas():
  texte_nettoye = nettoyer_texte(" BONJOUR monde ")
  assert nettoyer_texte(texte_nettoye) == texte_nettoye
```

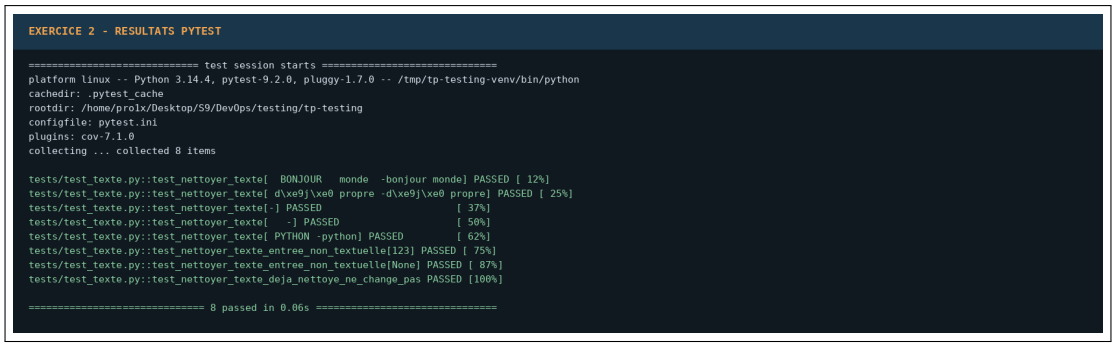


FIG. 2 : Sortie de l'exercice 2

Interprétation des résultats. Les tests confirment la conversion en minuscules, la réduction des espaces et la stabilité d'un texte déjà nettoyé, y compris pour une chaîne vide. Les entrées qui ne sont pas des chaînes doivent lever `TypeError`. En cas d'échec, comparer la sortie exacte attendue et vérifier que les entrées invalides ne sont pas acceptées silencieusement. La suite de ce fichier exécute huit cas : cinq valeurs paramétrées, deux entrées non textuelles et un test d'idempotence.

Exercice 3 | Fixtures et tests d'intégration

`app/reservation.py`

```python
class Stock:
  def __init__(self, quantite):
    self.quantite = quantite

  def retirer(self, quantite):
    if quantite <= 0:
      raise ValueError("Quantité positive requise")
    if quantite > self.quantite:
      return False
    self.quantite -= quantite
    return True


class Reservation:
  def __init__(self, stock):
    self.stock = stock

  def reserver(self, quantite):
    if self.stock.retirer(quantite):
      return "confirmee"
    return "refusee"
```

`tests/test_reservation.py`

```python
import pytest

from app.reservation import Reservation, Stock

```

```python

@pytest.fixture
def stock():
  return Stock(5)


def test_reservation_deux_unites_confirmee_et_stock_reduit(stock):
  reservation = Reservation(stock)
  assert reservation.reserver(2) == "confirmee"
  assert stock.quantite == 3


def test_reservation_six_unites_refusee_et_stock_inchange(stock):
  reservation = Reservation(stock)
  assert reservation.reserver(6) == "refusee"
  assert stock.quantite == 5


def test_reservation_cinq_unites_confirmee_et_stock_vide(stock):
  reservation = Reservation(stock)
  assert reservation.reserver(5) == "confirmee"
  assert stock.quantite == 0


def test_reservation_zero_unite_leve_value_error_et_stock_inchange(stock):
  reservation = Reservation(stock)
  with pytest.raises(ValueError):
    reservation.reserver(0)
  assert stock.quantite == 5
```

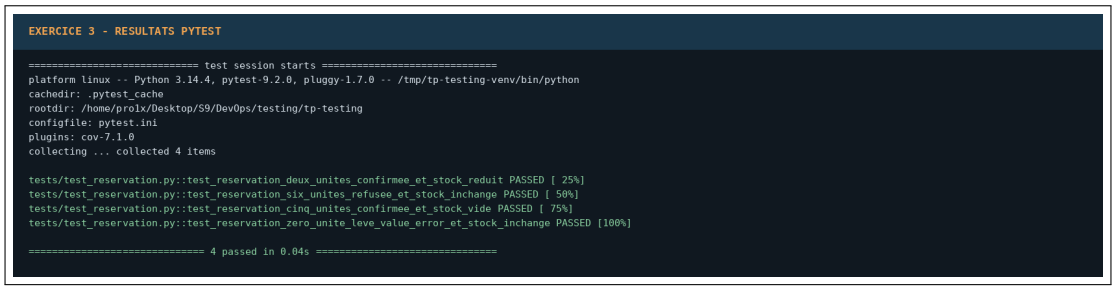


FIG. 3 : Sortie de l'exercice 3

Interprétation des résultats. La fixture fournit un stock neuf à chaque test. Une réservation valide réduit le stock, une quantité insuffisante est refusée sans le modifier et une quantité non positive lève `ValueError`. Si un test échoue, contrôler à la fois la décision renvoyée et la quantité restante : une réponse correcte avec un stock incorrect révèle tout de même un défaut d'intégration.

Réponses aux questions. Les résultats restent identiques quel que soit l'ordre d'exécution, car la fixture crée un nouveau `Stock(5)` pour chaque test : aucun état mutable n'est partagé. Ces tests vérifient une intégration parce qu'ils font collaborer les vrais objets `Reservation` et `Stock` et contrôlent à la fois le résultat et l'état du stock, sans remplacer de dépendance par un mock.

Exercice 4 | Isolation des dépendances

app/moderation.py

```python
def decider(modele, message):
  classe = modele.predict(message)
  if classe == "spam":
    return "bloquer"
  return "autoriser"
```

tests/test_moderation.py

```python
from unittest.mock import Mock

import pytest

from app.moderation import decider


def test_decider_bloque_un_spam():
  modele = Mock()
  modele.predict.return_value = "spam"
  assert decider(modele, "Offre miracle") == "bloquer"
  modele.predict.assert_called_once_with("Offre miracle")


def test_decider_autorise_un_message_normal():
  modele = Mock()
  modele.predict.return_value = "normal"
  assert decider(modele, "Bonjour") == "autoriser"
  modele.predict.assert_called_once_with("Bonjour")


def test_decider_propage_erreur_du_modele():
  modele = Mock()
  modele.predict.side_effect = RuntimeError("Modele indisponible")
  with pytest.raises(RuntimeError, match="Modele indisponible"):
    decider(modele, "Offre miracle")
```

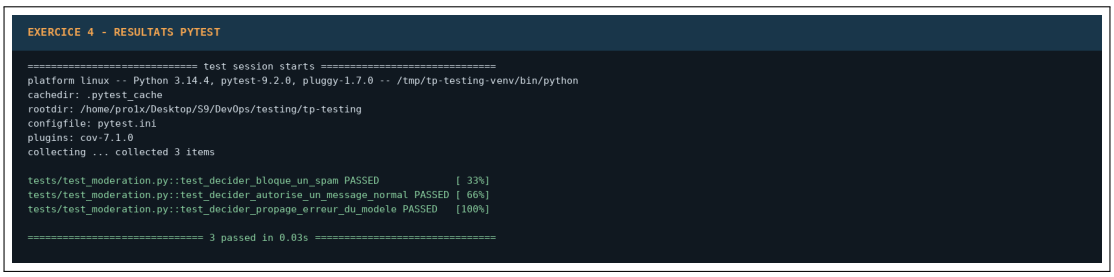


FIG. 4 : Sortie de l'exercice 4

Interprétation des résultats. Le mock permet de vérifier que `predict` reçoit le message une seule fois et que la décision correspond à sa réponse. Le test d'erreur confirme que l'exception du modèle est propagée. Si un test échoue, vérifier l'appel à la dépendance, la correspondance entre « spam » et « bloquer », et l'absence de masquage de l'erreur.

Réponse à la question. Ces tests valident le branchement entre la classe retournée par `predict` et la décision, ainsi que l'appel effectué avec le bon message. Ils ne permettent pas de conclure qu'un vrai modèle classe correctement les messages : le mock ne charge aucun modèle et ne mesure ni sa précision ni son comportement sur des données réelles.

Exercice 5 | Vérifier une alerte sans envoyer de notification

`app/notifications.py`

```python
def envoyer_notification(message):
    raise RuntimeError("Service non configure")
```

`app/surveillance.py`

```python
from app.notifications import envoyer_notification


def surveiller(taux_erreur):
    if taux_erreur > 0.10:
        envoyer_notification("Taux eleve")
        return "alerte"
    return "ok"
```

`tests/test_surveillance.py`

```python
from unittest.mock import patch

import pytest

from app.notifications import envoyer_notification
from app.surveillance import surveiller


def test_envoyer_notification_indisponible_leve_runtime_error():
    with pytest.raises(RuntimeError, match="Service non configure"):
        envoyer_notification("Taux eleve")


def test_surveiller_taux_superieur_au_seuil_declenche_alerte():
    with patch("app.surveillance.envoyer_notification") as notifier:
        assert surveiller(0.15) == "alerte"
    notifier.assert_called_once_with("Taux eleve")


def test_surveiller_taux_inferieur_au_seuil_ne_declenche_pas_alerte():
    with patch("app.surveillance.envoyer_notification") as notifier:
        assert surveiller(0.05) == "ok"
    notifier.assert_not_called()


def test_surveiller_taux_egal_au_seuil_ne_declenche_pas_alerte():
    with patch("app.surveillance.envoyer_notification") as notifier:
        assert surveiller(0.10) == "ok"
```

```
    notifier.assert_not_called()


def test_surveiller_propage_erreur_du_service_de_notification():
    with patch(
        "app.surveillance.envoyer_notification",
        side_effect=RuntimeError("Service indisponible"),
    ):
        with pytest.raises(RuntimeError, match="Service indisponible"):
            surveiller(0.15)
```

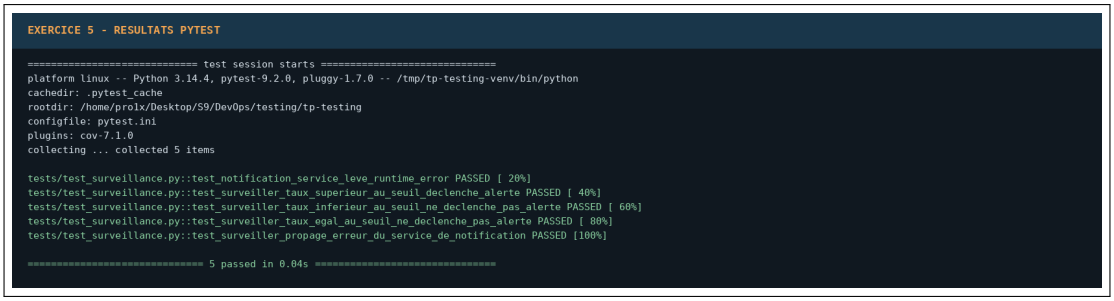


FIG. 5 : Sortie des tests de l'exercice 5

Interprétation des résultats. À 0,15, la fonction renvoie « alerte » et appelle une fois le service ; à 0,05 et au seuil exact 0,10, elle renvoie « ok » sans appel. Le test `side_effect` confirme que `RuntimeError` est propagée. Le patch cible `app.surveillance.envoyer_notification`, car c'est dans l'espace de noms du module testé que cette fonction importée est recherchée. En cas d'échec, vérifier le seuil strict, l'appel et son message, puis la propagation de l'exception. Un test supplémentaire vérifie aussi que le service de démonstration lève bien `RuntimeError` au lieu d'envoyer une notification.

Exercice 6 | Modérer un message via une requête

`app/web.py`

```
from flask import Flask, jsonify, request


def create_app():
    app = Flask(__name__)

    @app.post("/moderate")
    def moderate():
        data = request.get_json(silent=True)
        if not isinstance(data, dict):
            return jsonify(error="JSON object required"), 400

        message = data.get("message")
        if not isinstance(message, str) or not message.strip():
            return jsonify(error="Invalid message"), 400

        mots = message.lower().split()
        decision = "bloquer" if "spam" in mots else "autoriser"
```

```
        return jsonify(decision=decision), 200

    return app
```

tests/test_web.py

```python
import pytest

from app.web import create_app


@pytest.fixture
def client():
    app = create_app()
    app.config["TESTING"] = True
    return app.test_client()


def test_moderate_autorise_message_normal(client):
    response = client.post("/moderate", json={"message": "bonjour"})
    assert response.status_code == 200
    assert response.get_json() == {"decision": "autoriser"}


def test_moderate_bloque_message_contenant_spam_sans_casse(client):
    response = client.post("/moderate", json={"message": "ceci est du SPAM"})
    assert response.status_code == 200
    assert response.get_json() == {"decision": "bloquer"}


@pytest.mark.parametrize(
    "payload",
    [{}, {"message": 42}, {"message": " "}],
)
def test_moderate_refuse_message_invalide(client, payload):
    response = client.post("/moderate", json=payload)
    assert response.status_code == 400
    assert response.get_json() == {"error": "Invalid message"}


def test_moderate_refuse_corps_json_qui_n_est_pas_un_objet(client):
    response = client.post("/moderate", json=["message", "bonjour"])
    assert response.status_code == 400
    assert response.get_json() == {"error": "JSON object required"}
```



FIG. 6 : Sortie des tests de l'exercice 6

Interprétation des résultats. Les requêtes valides reçoivent le statut 200 et une décision JSON : « bonjour » est autorisé, tandis que le mot « SPAM » est bloqué sans tenir compte de la casse. Les données manquantes, de mauvais type ou vides reçoivent le statut 400 avec une erreur explicite ; un corps JSON qui n'est pas un objet reçoit également 400. En cas d'échec, vérifier à la fois le statut HTTP et le contenu JSON, pas seulement la décision interne. Le client Flask exerce la route et sa réponse sans démarrer de serveur réseau : il s'agit de tests fonctionnels de l'API, et non d'un parcours E2E complet.

Exercice 7 | Couverture de code

La configuration pytest ignore les tests de livraison pour les exercices suivants. Pour mesurer la suite complète, y compris les six tests de livraison, la commande ci-dessous remplace cette option et mesure aussi les branches :

```
python -m pytest -o addopts="" --cov=app --cov-branch \
  --cov-report=term-missing \
  --cov-report=html:rapport/coverage_html tests/
```

Question 1 : quel comportement manquait ? Avant le test supplémentaire, 32 tests réussissent et la couverture des instructions est de 99 %. Une ligne de `app/notifications.py` reste non couverte : l'appel réel au service n'était pas testé, car les tests de surveillance remplacent cette dépendance avec `patch`. Le test `test_notification_service_leve_runtime_error` a été ajouté pour vérifier que ce service de démonstration lève `RuntimeError` au lieu d'envoyer une notification.

COUVERTURE AVANT LE TEST SUPPLEMENTAIRE

Name	Stmts	Miss	Branch	BrPart	Cover	Missing
app/__init__.py	0	0	0	0	100%	
app/livraison.py	6	0	4	0	100%	
app/moderation.py	5	0	2	0	100%	
app/notifications.py	2	1	0	0	50%	2
app/reservation.py	17	0	6	0	100%	
app/surveillance.py	6	0	2	0	100%	
app/texte.py	4	0	2	0	100%	
app/web.py	15	0	4	0	100%	
TOTAL	55	1	20	0	99%	

Fig. 7 : Couverture des tests avant l'ajout du test du service de notification

Après cet ajout, les 33 tests réussissent. La couverture passe de 99 % à 100 % des instructions ; les 20 branches mesurées sont également couvertes (aucune branche partielle). Le rapport HTML confirme visuellement que les deux lignes de `app/notifications.py` sont exécutées.

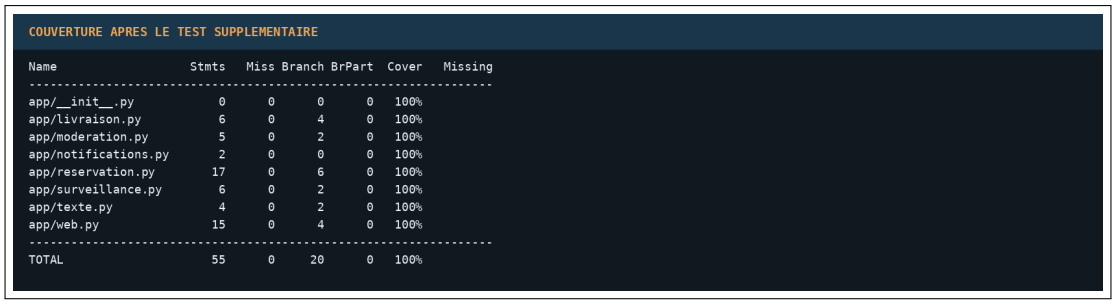


Fig. 8 : Couverture après l'ajout du test supplémentaire

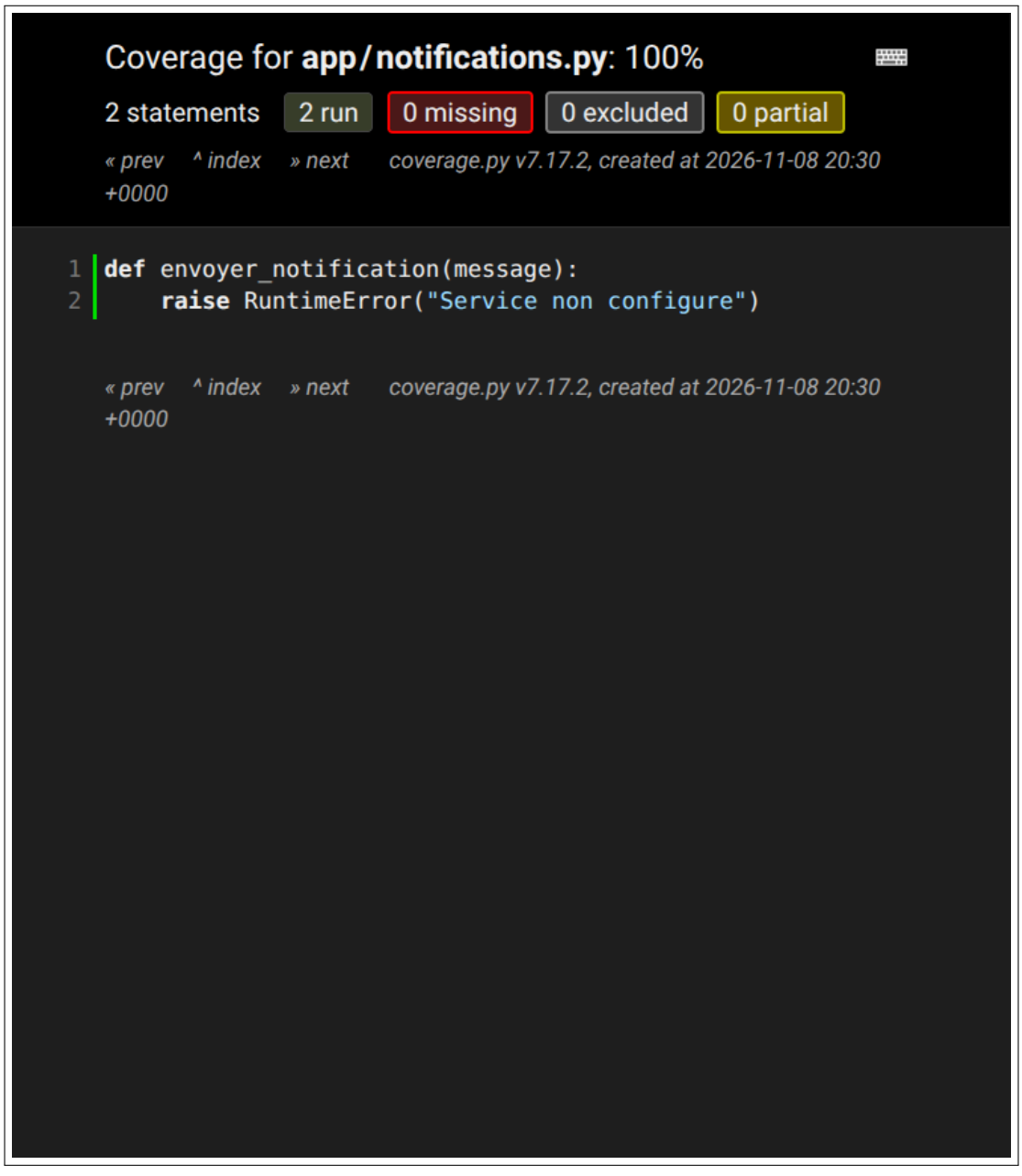


Fig. 9 : Rapport HTML détaillé : lignes exécutées dans app/notifications.py

Question 2 : comment interpréter l'évolution et ses limites ? Après cet ajout, les 33 tests réussissent. La couverture des instructions passe de 99 % à 100 % et les 20 branches mesurées

sont toutes parcourues. Cette progression vient d'un seul comportement manquant, désormais vérifié par une assertion sur l'exception attendue. Une couverture de 100 % ne garantit pas que tous les scénarios métier sont présents ni que les assertions détectent toutes les erreurs. Les tests de l'API, par exemple, ne remplacent pas une validation complète sur un vrai serveur ou en production.